

Komplexität & P vs. NP

Markus Hecher

Universität Potsdam, Germany

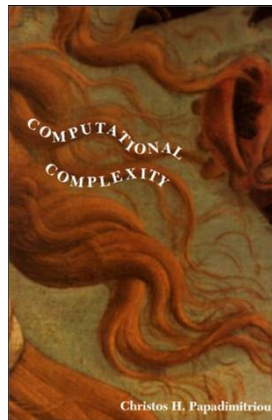
Vorlesung am June 15, 2026

Last updated: 2026-06-15

Komplexität von Problemen und Algorithmen

Background

- Christos H. Papadimitriou:
Computational Complexity.
Addison Wesley, 1994.



Komplexität von Problemen und Algorithmen

Scope

Komplexitätstheorie

Die **Komplexitätstheorie** beschäftigt sich mit der Analyse der “computational complexity” von **Problemen**. Konkreter: Wie viel **Laufzeit (Anzahl der Operationen)** oder **Speicherplatz (space)** ist notwendig, um ein bestimmtes Problem zu lösen?

Definition:

- **Problem**: besteht aus einer unendliche Menge an möglichen Instanzen und einer zugehörigen Frage
- **Entscheidungsproblem**: ist ein Problem, sodass die zugehörige Frage eine Ja/Nein Antwort hat

Beispielproblem **REACHABILITY**:

INSTANZ: Graph (V, E) und Knoten $u, v \in V$.

FRAGE: Gibt es einen Pfad im Graph von u nach v ?

- Andere Problemarten: **Funktionsprobleme**, **Optimierungsprobleme**, **Aufzählungsproblem**, **Zählprobleme**, ...

Algorithmus für REACHABILITY

REACHABILITY

INSTANZ: Graph (V, E) und Knoten $u, v \in V$.

FRAGE: Gibt es einen Pfad im Graph von u nach v ?

$S := \{u\};$

repeat

$S' := S;$

for all $i \in S'$ **do** {

for all $j \in V$ **do** {

if $(i, j) \in E$ **then** $S := S \cup \{j\};$

 }

 }

until $S = S';$

if $v \in S$ **then** return *true*

else return *false*;

Analyse

Wie effizient ist der Algorithmus?

Wie ist die Effizienz beeinflusst von

- Programmiersprache?
- Computerarchitektur?
- Eingaberepräsentation?

Effizienz des Algorithmus für REACHABILITY

- Eine naive Implementierung braucht kubische Zeit.
- Unter bestimmten Annahmen kann das Problem aber in Linearzeit in der Anzahl der Kanten gelöst werden.

Laufzeit und Speicherplatz

Wir messen die Komplexität von Problemen und Effizienz von Algorithmen mittels **Laufzeit** und **Speicherplatz**:

Gegeben Programm Π und Eingabe I für Π , die **Laufzeit von Π auf I** ist die Anzahl von atomaren Operationen, ausgeführt während des Laufes von Π auf Eingabe I .

↪ Die Definition von “atomarer Operation” hängt von der Definition der Programmiersprache und Computerarchitektur ab.

Gegeben Programm Π und Eingabe I , der **erforderliche Speicherplatz von Π auf I** ist die Anzahl von Bits an Speicher die verwendet wird während des Laufs von Π auf I .

Laufzeit und Speicherplatz II

Frage

Beeinflusst eine spezielle Programmiersprache oder Architektur Laufzeit/Speicherplatz einer Implementierung?

- Ja, aber nur marginal: eine spezielle Wahl von Sprache/Architektur gibt nur eine **geringe** Verbesserung an Laufzeit/Speicherplatz.
- Intuitiv “geringe Verbesserung” bedeutet:

Ein effizientes Programm kann in jede andere sinnvolle Programmiersprache und für jede andere sinnvolle Architektur übersetzt werden (zum Beispiel While-Programm), ohne Effizienz zu verlieren (unter Church-Turing These).

- ↪ Um die Existenz/Nichtexistenz effizienter Algorithmen zu untersuchen, genügen While-Programme oder einfacher Pseudocode (**unter Church-Turing These**).
- Formaler: “gering” in diesem Kontext heißt **polynomiell**.

Church-Turing Thesis Revisited

Konsequenz von Church-Turing These

Jeder Algorithmus kann mit einem While-Programm erstellt werden.

Alternative Formulierung: Jedes “vernünftige” **mathematische Modell eines Algorithmus** ist **äquivalent** zu dem eines While-Programms.

Stärkere Church-Turing These

Jedes “vernünftige” **mathematische Modell von Algorithmen und deren Zeit-/Speicherplatz-Performance** entspricht einem Modell und assoziierten Zeit-/Platz-Kosten, die **äquivalent** zu jenen eines While-Programms sind (modulo **polynomiellen Overhead**).

↪ Unter starker Church-Turing These genügt uns daher **einfacher Pseudocode** (mit üblichen Zuweisungen, Kontrollstrukturen, usw.).

Komplexität von Problemen und Algorithmen

Definition

Komplexität von Algorithmen

Wir interessieren uns für die **asymptotische, worst-case** Komplexität eines Algorithmus, also eine Funktion $f()$, sodass für jede Probleminstance der Größe n , die Antwort in Zeit/mit Speicherplatz $O(f(n))$ berechnet werden kann.

Recall: Wachstum

Seien $f, g: \mathbb{N} \rightarrow \mathbb{N}$.

- $f(n) = O(g(n))$ (**f wächst wie g oder langsamer**), wenn es $c, n_0 \in \mathbb{N}$ and n_0 such that for all $n \geq n_0$, $f(n) \leq c \cdot g(n)$
- $f(n) = \Omega(g(n))$, wenn $g(n) = O(f(n))$
- $f(n) = \Theta(g(n))$, wenn $g(n) = O(f(n))$ und $f(n) = O(g(n))$.

Konsequenz

Wir sind nicht interessiert an average-case, best-case, “Instanzen die typischerweise in der Praxis vorkommen”, Komplexität einer endlichen Anzahl an Instanzen, ...

Komplexität eines Problems

Komplexität eines Problems =
worst-case Komplexität des best-möglichen Algorithmus für dieses Problem

Komplexitätsanalyse

- klassifiziert Probleme anhand deren Komplexität
- analysiert die Komplexität von Problemen

Komplexitätstheorie

- untersucht Eigenschaften von Komplexitätsklassen
- studiert Relationen zwischen diesen Klassen

⇒ **Negative Resultate** (wie zum Beispiel “es gibt keinen effizienten Algorithmus für dieses Problem”) **brauchen formale Methoden**, wie zum Beispiel Reduktionen.

Komplexität von Problemen und Algorithmen

Fundamentale Komplexitätsklasse P

Komplexitätsklasse P (Polynomialzeit-Probleme)

Klasse P ist die Menge an Problemen, die in **polynomieller Laufzeit** in deren **Eingabegröße** gelöst werden können.

Die Klasse P besteht aus allen Entscheidungsproblemen $\mathcal{P}$, die Folgendes erfüllen:

- 1 es gibt ein (deterministisches!) Programm Π das $\mathcal{P}$ entscheidet und Π is derart, dass
- 2 für alle Instanzen I von $\mathcal{P}$ ist die Laufzeit von Π auf I polynomiell in $|I|$, d.h., in $O(|I|^k)$ für Konstante k .

Viele Probleme sind in P:

- **REACHABILITY** kann in kubischer Zeit gelöst werden und ist daher in P.
- Prüfen, ob ein String von einer kontextfreien Sprache erfasst wird, ist in P.

Probleme in P

Boolean Circuits

Ein Boolescher Schaltkreis (von Arität k) ist ein DAG (direkter azyklischer Graph) bestehend aus: k **Eingabe** Gatter (gates) $in_1, \dots, in_k$, ein **Ausgabe** Gatter und eine beliebige Anzahl an **AND**, **OR** und **NOT** Gatter.

- Eingabegatter haben 0 Kindgatter,
- NOT- und das Ausgabegatter haben 1 Kind,
- AND- und OR-Gatter haben zumindest 2 Kinder.

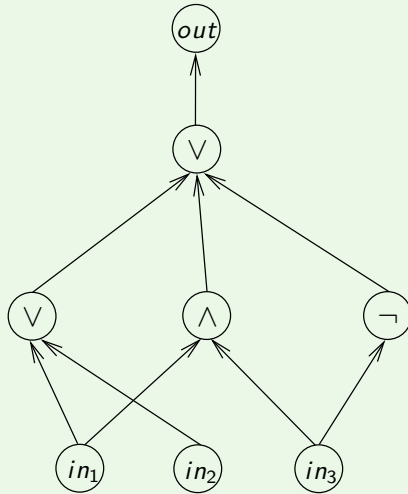
CIRCUIT-EVAL

INSTANZ: Boolescher Schaltkreis C und eine Funktion μ die jedem Eingabegatter von C den Wert *true* oder *false* zuweist.

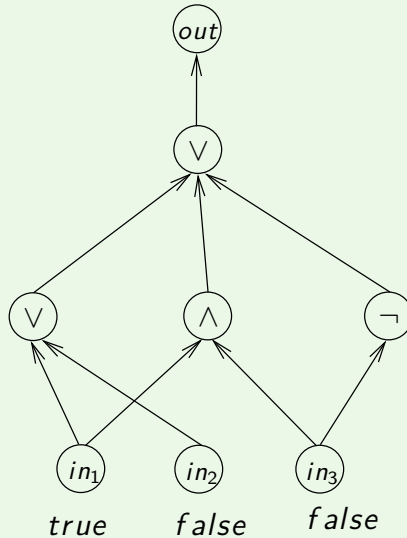
FRAGE: Wertet C unter gegebener (Wahrheits-)Belegung μ zu *true* aus?

(Belegung wird propagiert wie erwartet, siehe folgendes Beispiel).

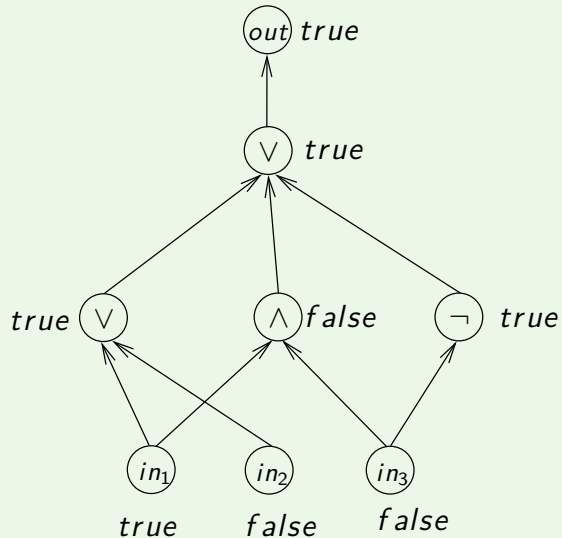
Beispiel: Schaltkreis



Beispiel: Schaltkreis mit Belegung



Beispiel: ausgewerteter Schaltkreis



CIRCUIT-EVAL ist in P

Algorithmus **evaluateCircuit**

Eingabe: Boolescher Schaltkreis der Arität k , Belegung μ

Ausgabe: $true$ gdw. C wertet zu $true$ aus unter μ

1 Sei $A := \{(in_1, \mu(in_1)), \dots, (in_k, \mu(in_k))\}$; /* Kopie von μ */

2 Sei G die Mengen aller Gatter in C ;

Die Knoten $g \in G$, die in A vorkommen werden, nennen wir ausgewertet.

3 **while** es gibt $g \in G$ sodass g nicht ausgewertet **do**

(i) Wähle $g \in G$, sodass g nicht ausgewertet, aber alle Kindgatter ausgewertet sind.
(So ein Gatter muss es geben, wenn es nicht ausgewertete Gatter gibt, weil C ein zusammenhängender DAG ist; nur Eingabegatter sind Quellen.)

(ii) Füge zu A das Tupel (g, v) hinzu, wobei Boolescher Wert v entsprechend dem Typ von g und den ausgewerteten Kindgatter-Werten von g (in A) bestimmt wird.

4 **if** $(out, true) \in A$ **then return true else return false**

CIRCUIT-EVAL ist in P II

Wir überprüfen, dass **evaluateCircuit** in Polynomialzeit in der Eingabegröße läuft:

- 1 Relation A ist immer linear in der Größe von C .
- 2 Dadurch, dass jede Iteration der Schritte (i-ii) ein neues ausgewertetes Gatter erzeugt, gibt es maximal $|G|$ Durchgänge der äußeren **while** Schleife.
- 3 Die Überprüfung, ob ein Gatter $g \in G$ nicht ausgewertet ist und all dessen Eingabegatter nicht ausgewertet sind, läuft in quadratischer Zeit in der Größe von G (bis zu $|G|$ Mal A konsultieren).
- 4 Wegen Beobachtung (3), ist Schritt (i) in kubischer Zeit machbar (traversiere G und prüfe wie oben beschrieben).
- 5 Schritt (ii) ist linear in der Größe von C .

↪ Der Algorithmus läuft in $O(n^4)$.

Man kann sogar $O(n)$ anstelle von $O(n^4)$ zeigen.

Diskussion

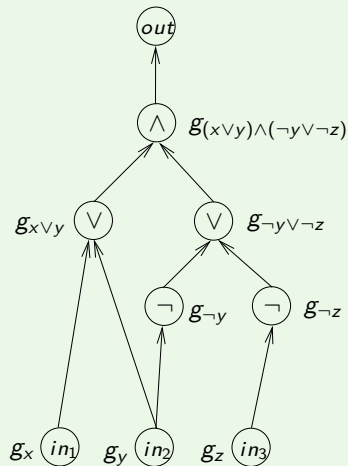
- Ein Problem ist **tractable** (effizient lösbar, handhabbar), wenn es in der Klasse P ist. Recall: Dann gibt es einen Algorithmus, der das Problem in Polynomialzeit in der Größe n der Eingabe löst, d.h. in $O(n^k)$.
- ↪ Hier nur **asymptotische, worst-case Komplexität**.
- Es gibt genug praktische Beispiele, wo ein andere Komplexitätsbegriff angebracht wäre, zB. Linear Programming (LP).
 - Es gibt Algorithmen mit polynomieller worst-case Komplexität.
 - ↪ **Erfahrung**: Simplex (exponentieller Worst Case) funktioniert praktisch schnell.
- Gibt es keinen Polynomialzeit-Algorithmus für ein Problem, ist das Problem **intractable** (nicht handhabbar).
 - **intractability** (kein effizienter Algorithmus bekannt und dessen Existenz ist sehr unwahrscheinlich – obwohl formal möglich)
 - Klasse NP als Nächstes
 - **provable intractability** (Existenz eines effizienten Algorithmus formal ausgeschlossen)
 - Klasse EXPTIME

Zusammenhang zwischen Booleschen Formeln und Schaltkreisen

Beispiel:

$$\varphi = (x \vee y) \wedge (\neg y \vee \neg z)$$

$C(\varphi)$



Model Checking ist in P

MODEL-CHECKING

INSTANZ: Boolesche Formel φ und eine Belegung T

QUESTION: $T(\varphi) = \text{true}$?

$\leadsto$ Wir folgern, dass **MODEL-CHECKING** in P ist.

- Wie? Reduktion von **MODEL-CHECKING** auf **CIRCUIT-EVAL**, was in Polynomialzeit funktioniert!
- Wir wissen bereits, dass **CIRCUIT-EVAL** in P ist.

Satisfiability und Validity

- Eine Boolesche Formel φ ist **erfüllbar (satisfiable)** gdw. es eine Belegung T von $\text{var}(\varphi)$ gibt, sodass $T(\varphi) = \text{true}$.
- Eine Boolesche Formel φ ist **gültig (valid)** gdw. für jede Belegung T von $\text{var}(\varphi)$ gilt: $T(\varphi) = \text{true}$.

SAT

INSTANZ: Boolesche Formel φ .

FRAGE: Ist φ erfüllbar?

VALIDITY

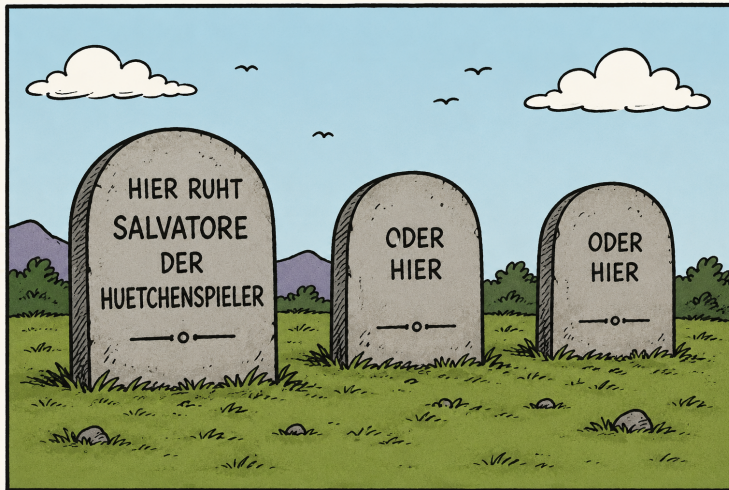
INSTANZ: Boolesche Formel φ .

FRAGE: Ist φ gültig?

Komplexität von Problemen und Algorithmen

Fundamentale Komplexitätsklasse NP

Die Klasse NP



Die Klasse NP

- Gegeben eine Formel φ und eine zugehörige Belegung T : damit können wir effizient verifizieren, ob φ unter T zu *true* auswertet.
 - Recall: **MODEL-CHECKING** ist in P.
- ↪ Erfüllbarkeit von φ entscheidbar, indem über alle möglichen Belegungen enumeriert und effizient überprüft wird, ob φ unter Belegung zu wahr auswertet.
- Welche Strategie? Wie gehen wir über die Belegungen? ↪ Schwierig zu sagen!
- Naives Aufzählen aller Belegungen benötigt exponentielle Laufzeit:
 - 2^n mögliche Belegungen in der Anzahl n der Variablen in φ .
- Mit “Glück” finden wir die richtige Belegung früh (in Polynomialzeit).
- ↪ “Glück” kann durch Nichtdeterminismus gewissermaßen erzwungen werden. NP heißt direkt übersetzt **Nicht-deterministisch Polynomialzeit**.

Die Klasse NP II

Wir benötigen Nichtdeterminismus!

↪ Erweiterung von Pseudocode-/While-Programmen um ein Konstrukt `choice(Option1,Option2)`. Intuition: `choice` spaltet die Berechnung in 2 (parallele) Zweige, einer mit Option1, der andere mit Option2.

↪ Wir nennen ein Pseudocode-Program mit `choice`-Anweisungen **choice-Programm**.

- Laufzeit eines `choice`-Programms ist das Maximum über alle Zweige (d.h., der Zweig der am Längsten Zeit benötigt.)
- `choice`-Programm ist wahr, wenn **zmd. eine Berechnung** wahr ist.

SAT via choice-Programm

```
Boolean SAT(String s, Integer n) /* Formel s über Variablen  $a_1, \dots, a_n$  */  
  for all  $1 \leq i \leq n$  do {  
    choice( $t_i = true, t_i = false$ );  
  }  
  return modelcheck (s, [ $t_1, \dots, t_n$ ]); /* prüfe ob s wahr unter Belegung  $a_i \mapsto t_i$  */
```

NP via Guess & Check I

We geben eine erste Definition für die Klasse NP.

NP ist die Menge der Entscheidungsprobleme $\mathcal{P}$ für die gilt:

- 1 es gibt ein choice-Programm Π das $\mathcal{P}$ entscheidet, sodass für Π gilt:
- 2 für alle Instanzen I von $\mathcal{P}$ ist die Laufzeit von Π auf I polynomiell in $|I|$, d.h. die Laufzeit ist in $O(|I|^k)$ für Konstante k .

↪ Jedes Problem in NP kann man via einer **Guess & Check** Prozedur charakterisieren

NP via Zertifikate

- Intuitiv: Eine Instanz I ist positiv für $\mathcal{P}$ gdw. wir ein korrektes “Zertifikat” (via choice statements) **raten können**.
 - Jedes Zertifikat bezeugt uns die Positivität von I und muss polynomiell groß sein!
- ↪ Probleme in NP kann man auch via **kompakter Zertifikate** analysieren: zB. Lösungen, Beweise, Zeugen (witnesses), etc.

Beispiel: **SAT**

- Wenn φ erfüllbar ist (d.h. es ist eine positive Instanz von **SAT**), dann gibt es eine Belegung (**kompaktes Zertifikat**), welche Erfüllbarkeit bezeugt.
- Wenn φ unerfüllbar ist (d.h. es ist eine negative instanz von **SAT**), gibt es kein solches Zertifikat.

NP via Zertifikate II

NP ist die Klasse der Entscheidungsprobleme $\mathcal{P}$, sodass:

- positive Instanzen I von $\mathcal{P}$ haben **kompakte Zertifikate**
- für gegebene Instanz I von $\mathcal{P}$ und Kandidatzertifikat C ist es **billig zu verifizieren**, ob C belegt/bezeugt, dass I eine positive Instanz von $\mathcal{P}$ ist.

↪ “kompakt” und “billig” meint “polynomiell in der Instanzgröße”!

Eine **Zertifikatsrelation** R für Entscheidungsproblem $\mathcal{P}$ ist $R \subseteq \text{INSTANZEN}(\mathcal{P}) \times \text{ZERT}$, wobei ZERT eine Menge endlicher Objekte ist, sodass

$I \in \text{INSTANZEN}(\mathcal{P})$ ist eine positive Instanz von $\mathcal{P}$

gdw

es gibt $C \in \text{ZERT}$ für das gilt $(I, C) \in R$.

NP via Zertifikate III

Wir formalisieren kurze und einfach-verifizierbare Zertifikate.

Eine binäre Relation R ist **polynomiell entscheidbar** when es einen Polynomialzeit-Algorithmus gibt, der für v_1, v_2 überprüft ob $(v_1, v_2) \in R$.

R ist **polynomiell balanciert** wenn $(v_1, v_2) \in R$ impliziert, dass $|v_2| \leq |v_1|^k$ für Konstante $k \geq 1$.

Dies liefert dann eine alternative Definition der Klasse NP.

Ein Entscheidungsproblem $\mathcal{P}$ ist in NP wenn es für $\mathcal{P}$ eine polynomiell balancierte und polynomiell entscheidbare Zertifikatsrelation gibt.

↪ Warum sind beiden Definitionen für NP gleich? (Übung)

Beispiel: **SAT** ist noch immer in **NP**

Theorem 1.

$SAT \in NP$.

Beweis.

Um zu beweisen, dass **SAT** $\in$ **NP**, definieren wir eine polynomiell balancierte und polynomiell entscheidbare Zertifikatsrelation, wie folgt.

$$R = \{(\varphi, \mu) \mid \text{Formel } \varphi \text{ wertet zu } true \text{ aus unter Belegung } \mu\}.$$

- R ist eine Zertifikatsrelation per Konstruktion: φ ist eine positive Instanz von **SAT** $\Leftrightarrow$ es gibt eine Belegung μ die φ zu *true* evaluieren lässt $\Leftrightarrow (\varphi, \mu) \in R$.
- R ist polynomiell balanciert, weil jede Belegung μ für φ als Teilmenge der Variablen $var(\varphi)$ der Formel repräsentiert werden kann.
- R ist polynomiell entscheidbar, weil die Auswertung von Formel φ unter μ in P ist.

Ausblick und Zusammenfassung

- Beobachtung: es gibt eine interessante Diskrepanz zwischen positiver und negativer Instanzen von NP-Problemen:
 - Eine positive Instanz kann in Polynomialzeit gelöst werden wenn wir **richtig raten**.
⇒ **Heuristiken sind hilfreich** (um den riesigen Suchraum zu navigieren).
 - Eine gute Strategie hilft nicht beim Lösen negativer Instanzen.
- Asymptotische worst-case Komplexität vs. anderer Definition
- Grundverständnis von asymptotischen Wachstum (polynomiell vs. exponentiell)
- Klasse P vs. Klasse NP
- Handhabbarkeit (Tractability) vs. Nichthandhabbarkeit (intractability)